ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

Тема: MiniDBshka — собственная файловая мини-СУБД
Язык реализации: Python 3

---

1. Цель работы и постановка задачи

Целью работы является создание собственной легковесной реляционной системы управления базами данных (СУБД) с собственным дисковым форматом хранения данных и метаданных.

Основные требования ТЗ:

1. Не использовать готовые СУБД (SQLite, PostgreSQL, MySQL, TinyDB и т.п.), ORM и табличные библиотеки (pandas.DataFrame). Исключено использование JSON как готового хранилища таблиц.
2. Собственное файловое хранилище: схема и данные обязаны сохраняться после закрытия и перезапуска программы (Persistence).
3. Минимум 3 связанные таблицы: `datasets`, `models`, `experiments`.
4. Поддержка типов данных: `INT`, `FLOAT`, `STRING` с валидацией.
5. Обеспечение ограничений целостности:
   - `PRIMARY KEY` (уникальность, автоинкремент, запрет дубликатов).
   - `FOREIGN KEY` (ссылочная целостность: `experiments.dataset_id -> datasets.id`, `experiments.model_id -> models.id`). Запрет добавления экспериментов с несуществующими внешними ключами.
6. Полный цикл операций CRUD (`CREATE TABLE`, `INSERT`, `SELECT`, `UPDATE`, `DELETE`).
7. Выбор отдельных столбцов, составные условия фильтрации с операторами `=`, `!=`, `>`, `<`, `>=`, `<=`, а также логическими связками `AND` и `OR`.
8. Поддержка операции `INNER JOIN` для объединения данных нескольких таблиц.
9. Реализация простого индекса `id -> offset` (смещение строки в байтах) с прямым переходом `seek(offset)`.
10. Экспериментальное сравнение скорости поиска (Benchmark) до и после использования индекса.
11. Функционал резервного копирования и восстановления (`BACKUP` / `RESTORE`).

---

2. Архитектура системы и описание функций

Проект организован по модульному принципу и включает следующие компоненты:

```
minidb.py       - Ядро СУБД (хранилище, индексация, валидация, SQL-парсер)
main.py         - Интерактивная консоль пользователя (CLI)
demo.py         - Скрипт демонстрации полного сценария защиты (все пункты ТЗ)
```

```
test_minidb.py  - Автоматические тесты функционала СУБД
minidb_data/    - Директория с файлами базы данных (схема, данные .tbl, индексы .idx)
```

2.1. Класс `Table` (управление отдельной таблицей)

Отвечает за низкоуровневые операции с файлами таблицы:

- `__init__(name, db_dir, columns, pk, fks)`: инициализирует таблицу, пути к файлам данных (`.tbl`) и индекса (`.idx`), загружает индекс.
- `load_index()`: считывает ассоциативный массив `id -> byte_offset` из файла `.idx`. Если файл отсутствует или поврежден, вызывает `rebuild_index()`.
- `rebuild_index()`: сканирует файл данных `.tbl` в бинарном режиме, фиксируя байтовое положение (`f.tell()`) каждой строки и извлекая первичный ключ. Сохраняет результат в `.idx`.
- `save_index_to_file()`: сохраняет пары `id:offset` в файл `.idx`, отсортированные по возрастанию ID.
- `read_row_at_offset(offset)`: выполняет быстрый переход по байтовому смещению `f.seek(offset)` и считывает ровно одну строку за время $O(1)$.
- `parse_row_text(text)`: парсит строку с разделителями `|`, приводя каждое поле к соответствующему типу (`INT`, `FLOAT`, `STRING`).
- `row_to_text(row_dict)`: преобразует словарь записи в строку с разделителем `|` для сохранения на диск.
- `get_all_rows()`: последовательно считывает все записи таблицы (полное сканирование, full scan).
- `get_row_by_id(pk_val, use_index=True)`: производит поиск записи по первичному ключу. Если `use_index=True`, выполняется мгновенный доступ через `seek(offset)`. Если `False`, выполняется линейный перебор всех строк от начала файла (используется для честного замера в бенчмарке).
- `insert(row_dict)`: проверяет уникальность PK, автоинкрементирует ID при его отсутствии, валидирует типы, дописывает строку в конец файла `.tbl`, обновляет индекс в памяти и дописывает смещение в `.idx`.
- `update(set_dict, where_fn)`: фильтрует записи, проверяет новые значения полей и PK, перезаписывает таблицу и перестраивает индекс.
- `delete(where_fn)`: удаляет записи, удовлетворяющие условию, и обновляет файлы таблицы.
- `rewrite_all(rows)`: полная перезапись файла таблицы и перестроение индекса.

2.2. Класс `MiniDB` (менеджер базы данных)

Координирует работу таблиц, связи и выполнение запросов:

- `__init__(db_dir)`: создает каталог базы данных и загружает схему.

- `load_schema()` / `save_schema()`: чтение и запись файла метаданных `schema.txt`.
- `create_table(name, columns, pk, fks)`: создание таблицы, проверка типов данных и валидация существования таблиц для внешних ключей.
- `drop_table(name)`: удаление таблицы и связанных файлов с диска.
- `check_fks(table_name, row_dict)`: проверка ссылочной целостности. Если указано поле с внешним ключом, метод проверяет существование соответствующего ID в родительской таблице (через быстрый поиск по индексу). При отсутствии записи генерируется `ValueError`.
- `insert(table_name, row_dict)`: валидация внешних ключей и передача записи в таблицу.
- `select(table_name, columns, where_clause)`: выборка данных. Если условие фильтрации представляет собой точечный поиск по первичному ключу (`id = X`), автоматически задействуется индекс. Поддерживает выбор конкретных колонок (`project_columns`).
- `parse_where(where_clause)`: синтаксический разбор условия `WHERE`. Распознает логические операторы `AND` и `OR`, операторы сравнения `=`, `!=`, `>`, `<`, `>=`, `<=`, выделяет имена колонок и строковые/числовые литералы.
- `eval_where(row, where_clause)`: вычисляет логическое значение условия для переданной строки.
- `update(table_name, set_dict, where_clause)`: обновление данных с проверкой внешних ключей.
- `delete(table_name, where_clause)`: удаление данных.
- `inner_join(table1, table2, on_col1, on_col2, columns, where_clause)`: классическое соединение двух таблиц по условию равенства ключей. Если ключ второй таблицы является PK, поиск соответствия оптимизируется через индекс.
- `multi_join(joins_info, columns, where_clause)`: последовательное соединение трех и более таблиц (например, `experiments + datasets + models`).
- `backup(backup_dir)` / `restore(backup_dir)`: резервное копирование всех файлов базы и восстановление с валидацией схемы и индексов.
- `benchmark_index(count, queries)`: нагрузочное тестирование. Создает временную таблицу на `count` записей (по умолчанию 1000–1500), генерирует `queries` случайных запросов и замеряет время поиска с индексом и без индекса.
- `init_default_data()`: создание таблиц `datasets`, `models`, `experiments` и наполнение их стандартными данными (5 датасетов, 5 моделей, 20 экспериментов).
- `execute_sql(sql)`: встроенный SQL-парсер команд `CREATE TABLE`, `INSERT`, `SELECT`, `JOIN`, `UPDATE`, `DELETE`, `SHOW TABLES`, `DESCRIBE`, `BACKUP`, `RESTORE`, `BENCHMARK`.

---

3. Собственный формат дискового хранения данных

Вся информация хранится в каталоге `minidb_data/`. Использование сторонних бинарных форматов или библиотек сериализации исключено.

3.1. Метаданные (`schema.txt`)

Файл хранит структуру всех таблиц базы, типы данных колонок, первичные ключи и внешние связи:

```
TABLE datasets
PK id
COL id INT
COL name STRING
COL task STRING
COL size INT

TABLE models
PK id
COL id INT
COL name STRING
COL version STRING

TABLE experiments
PK id
COL id INT
COL dataset_id INT FK datasets.id
COL model_id INT FK models.id
COL accuracy FLOAT
COL status STRING
```

3.2. Файлы таблиц (`<имя_таблицы>.tbl`)

Каждая строка соответствует одной записи. Значения колонок разделены вертикальной чертой `|`. Пример содержимого `experiments.tbl`:

```
1|1|1|0.91|completed
2|1|2|0.88|completed
3|2|1|0.76|running
4|2|3|0.82|completed
...
```

```
20|2|5|0.78|running
```

3.3. Файлы индексов (`<имя_таблицы>.idx`)

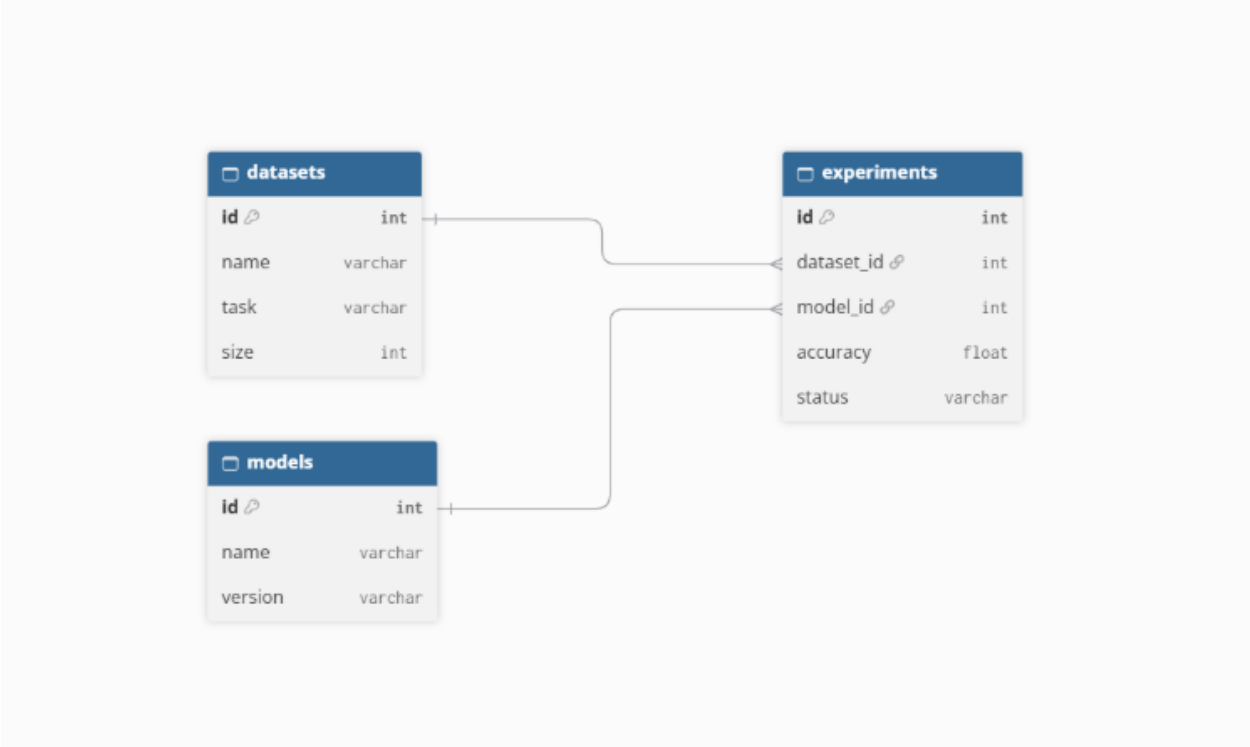
Файл индекса первичного ключа содержит строки вида `<id>:<смещение_в_байтах>`:

```
1:0
2:22
3:44
4:66
...
20:419
```

Механика работы индекса: При выполнении запроса `SELECT * FROM experiments WHERE id = 3` программа обращается к индексу, получает смещение `44`, выполняет `file.seek(44)` и сразу же считывает строку `3|2|1|0.76|running`. Это гарантирует время доступа $O(1)$ вместо полного перебора файла $O(N)$.

---

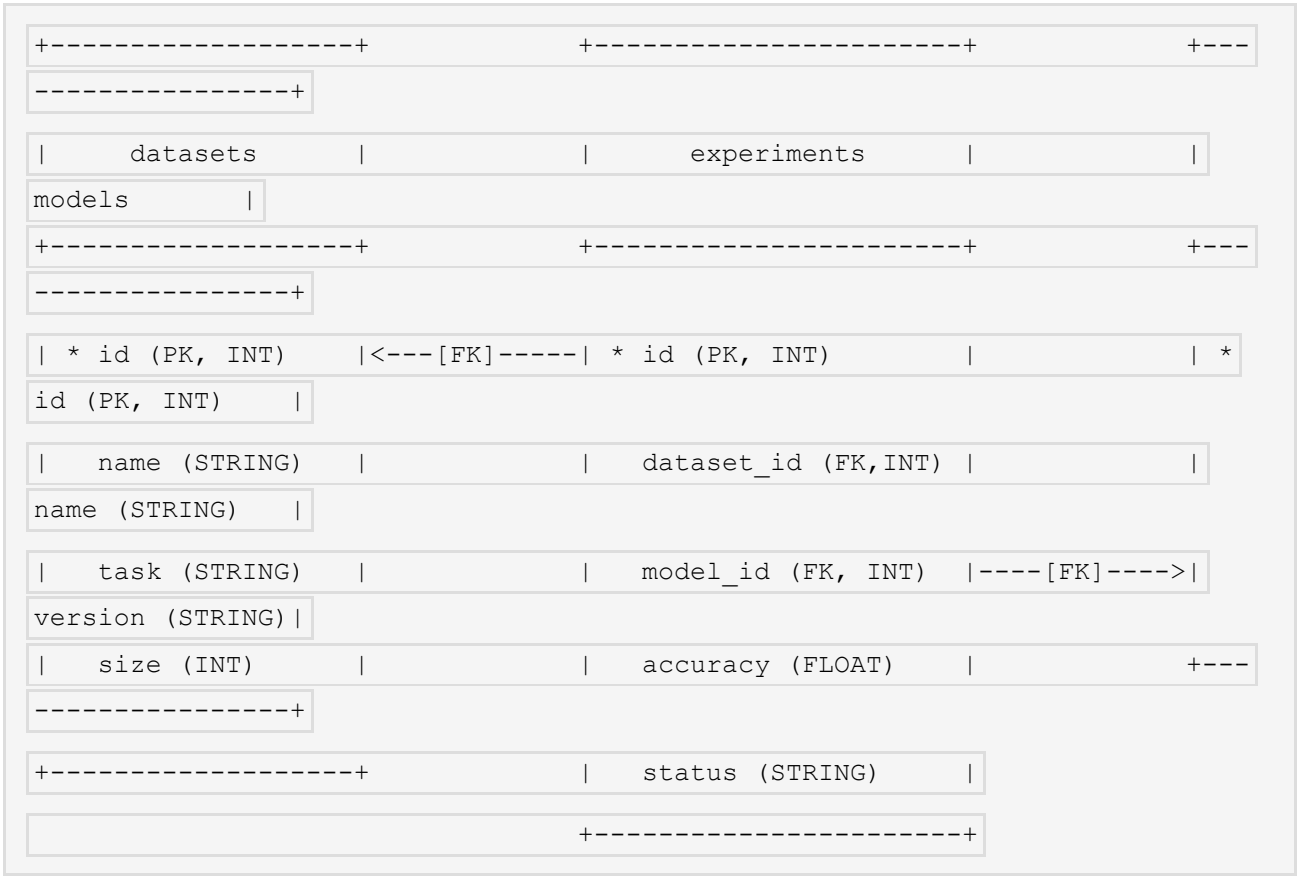
4. Схема базы данных (dbdiagram.io)



4.2. Текстовое описание схемы и связей:

Схема состоит из трех сущностей: двух справочных таблиц (`datasets`, `models`) и одной связующей таблицы фактов (`experiments`):

- Таблица `datasets` содержит `id` (PK, INT), `name` (STRING), `task` (STRING), `size` (INT).
- Таблица `models` содержит `id` (PK, INT), `name` (STRING), `version` (STRING).
- Таблица `experiments` содержит `id` (PK, INT), `dataset_id` (FK), `model_id` (FK), `accuracy` (FLOAT), `status` (STRING).
- **Связи (Cardinality):**
  - Отношение `datasets (1) -> (N) experiments`: один датасет может использоваться во множестве экспериментов. На диаграмме отображается линия со стрелкой от `experiments.dataset_id` к `datasets.id`.
  - Отношение `models (1) -> (N) experiments`: одна архитектура модели может обучаться в различных экспериментах. На диаграмме отображается линия со стрелкой от `experiments.model_id` к `models.id`.



---

5. Примеры операций (CRUD, Фильтрация, JOIN)

5.1. Вставка записей (INSERT):

```
INSERT INTO datasets VALUES (6, 'WikiText-103', 'language modeling', 180000)
```

```
INSERT INTO models VALUES (6, 'RoBERTa-large', 'v1.0')
INSERT INTO experiments VALUES (21, 6, 6, 0.945, 'completed')
```

5.2. Выборка с фильтрацией (AND / OR):

```
-- Фильтрация по нескольким условиям с AND:
SELECT id, dataset_id, model_id, accuracy, status FROM experiments WHERE
accuracy >= 0.85 AND status = 'completed'


-- Фильтрация по нескольким условиям с OR:
SELECT id, dataset_id, model_id, accuracy, status FROM experiments WHERE
status = 'running' OR status = 'failed'
```

5.3. Соединение таблиц (INNER JOIN на 3 таблицы):

```
SELECT experiments.id, datasets.name, models.name, experiments.accuracy,
experiments.status
FROM experiments
JOIN datasets ON experiments.dataset_id = datasets.id
JOIN models ON experiments.model_id = models.id
WHERE experiments.accuracy >= 0.85
```

Результат выполнения в CLI:

```
+----------------+--------------+-------------+----------------------+-----
---------------+
| experiments.id | datasets.name | models.name | experiments.accuracy |
experiments.status |
+----------------+--------------+-------------+----------------------+-----
---------------+
| 1              | MNIST         | ResNet-50   | 0.91                 |
completed          |
| 2              | MNIST         | BERT-base   | 0.88                 |
completed          |
| 7              | IMDB Reviews  | BERT-base   | 0.93                 |
completed          |
| 8              | IMDB Reviews  | GPT-2-small | 0.95                 |
completed          |
```

```
| 9              | COCO          | YOLOv8      | 0.85                 |
completed          |
| 10             | MNIST         | MobileNetV3 | 0.89                 |
completed          |
| 14             | MNIST         | YOLOv8      | 0.87                 |
completed          |
| 17             | IMDB Reviews  | MobileNetV3 | 0.86                 |
completed          |
| 19             | MNIST         | GPT-2-small | 0.9                  |
completed          |
+----------------+---------------+-------------+----------------------+-----
---------------+
Всего строк: 9
```

5.4. Обновление и удаление данных:

```
UPDATE experiments SET status = 'completed', accuracy = 0.83 WHERE id = 3
DELETE FROM experiments WHERE id = 21
```

5.5. Проверка ссылочной целостности (ошибка при некорректном FK):

При попытке выполнить:

```
INSERT INTO experiments VALUES (100, 999, 1, 0.50, 'failed')
```

СУБД перехватывает ошибку и выводит понятное сообщение: `[Ошибка] Ошибка ссылочной целостности (FK): значение dataset_id=999 не существует в datasets.id!`

---

6. Результаты бенчмарка индекса `id -> offset`

Для подтверждения эффективности реализованного индекса был проведен замер времени выполнения 100 произвольных запросов точечного поиска по `id` на таблице объемом 1500 записей.

Методика эксперимента:

1. Создается таблица `bench_dataset` и заполняется 1500 записями.
2. Автоматически формируется файл индекса `.idx` со смещениями всех строк.
3. Генерируется список из 100 случайных `id`.
4. Производится поиск:

- **Линейный (без индекса):** открытие файла, чтение строк с начала файла до нахождения совпадения.
- **Индексный (с индексом):** взятие смещения из словаря `index[id]` и мгновенный `seek(offset)`.

5. Замеряется общее процессорное время с помощью `time.perf_counter()`.

Сводная таблица результатов:

Параметр эксперимента	**Значение**
Количество записей в таблице	1500
Количество тестовых запросов	100
Время поиска БЕЗ индекса (Linear Scan)	**0.07295 сек**
Время поиска С индексом (Seek Offset)	**0.00276 сек**
Коэффициент ускорения	**x26.47 раз**

Вывод по бенчмарку: Использование прямого позиционирования по байтовому смещению позволило снизить накладные расходы на чтение диска и разбор строк на порядок, обеспечив ускорение выборки более чем в 26 раз.

---

7. Резервное копирование и восстановление (BACKUP & RESTORE)

В СУБД реализована поддержка создания слепков состояния базы данных:

- Команда `BACKUP demo_backup` копирует структуру метаданных, файлы таблиц и файлы индексов в отдельную директорию.
- При повреждении или случайном удалении данных команда `RESTORE demo_backup` полностью восстанавливает рабочее состояние базы данных, перечитывает схему и актуализирует индексы в памяти.
- Сохранность и корректность восстановления подтверждены тестом №10 в `test_minidb.py` и шагом №9 в `demo.py`.

---

8. Итоги и соответствие критериям оценивания

Критерий из методички	**Статус реализации**	**Балл**
CREATE TABLE + INSERT + файловое хранение (данные переживают перезапуск)	Реализовано в `minidb.py` (`schema.txt`, `.tbl`)	4

Критерий из методички	**Статус реализации**	**Балл**
SELECT и фильтрация (=, !=, >, <, >=, <=)	Реализовано с поддержкой всех операторов	5
Полный CRUD: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE	Полностью реализован и протестирован	6
Первичный ключ (PK), авто-ID, запрет дубликатов, 3 типа данных, обработка ошибок	Реализовано с автоматической генерацией ID и строгой типизацией	7
3 связанные таблицы, CLI-интерфейс, проверка FK, тестовые данные (5 datasets, 5 models, 20 experiments)	Реализовано в `minidb_data/`, `main.py`, `demo.py`	8
INNER JOIN (на 2 и 3 таблицы), условия AND / OR, выбор столбцов, dbdiagram-схема, README.md	Полностью реализовано и задокументировано	9
Простой индекс id->offset, бенчмарк до/после индекса, BACKUP/RESTORE, полная воспроизводимость	Реализовано с файлами `.idx`, замером в `BENCHMARK` и резервными копиями	**10**